Выполнил: Василенко И.А

Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №2

Дисциплина: Основы программирования на Python

Тема: Коллекции, строки и функции в Python

Вариант 2: Зарплата сотрудников

1. Цель работы

Получить практические навыки работы с основными структурами данных языка Python и научиться применять их для обработки наборов информации. Освоить работу со списками, словарями, множествами, циклами, функциями с параметрами и интерактивным меню.

2. Постановка задачи

Разработать консольное приложение для автоматизации работы с данными сотрудников. Для каждого сотрудника необходимо хранить: ФИО, должность, отдел и зарплату.

Программа должна реализовывать следующие операции:

1) Добавление нового сотрудника с клавиатуры с проверкой ввода.
2) Вывод списка всех сотрудников.
3) Поиск сотрудников по заданному отделу.
4) Фильтрация сотрудников по зарплате (вывод тех, у кого з/п выше или равна порогу).
5) Определение статистических показателей (средняя зарплата, самый высокооплачиваемый сотрудник).
6) Вывод списка уникальных должностей в компании.
7) Сортировка сотрудников по уровню зарплаты.

3. Описание алгоритма и структуры данных

В качестве основной структуры хранения данных используется список словарей (list из dict). Такая структура позволяет удобно связывать параметры каждого объекта и динамически расширять базу данных.

Для реализации логики задачи были разработаны следующие пользовательские функции, получающие данные через параметры (без использования глобальных переменных):

1) add_worker(spisok) — запрашивает данные у пользователя, проверяет корректность ввода зарплаты через конструкцию try-except и добавляет словарь в общий список.

2) show_all(spisok) — перебирает коллекцию циклом for и красиво выводит данные каждого сотрудника.

3) poisk_it(spisok) — осуществляет поиск сотрудников по совпадению названия отдела. Изучен алгоритм работы с флагом для уведомления об отсутствии результатов.

4) filtr_zp(spisok) — фильтрует числовые значения зарплат по критерию «больше или равно».

5) sr_zp(spisok) — рассчитывает среднее арифметическое всех зарплат с использованием цикла накопления суммы и функции len().

6) samiy_cool(spisok) — реализует классический алгоритм поиска максимума в цикле.

7) unique_positions(spisok) — собирает должности в коллекцию set (множество) для автоматического удаления дубликатов.

8) sort_zp(spisok) — выполняет сортировку коллекции с помощью встроенной функции sorted() и лямбда-выражения в качестве ключа.

9) Взаимодействие с пользователем организовано через интерактивное текстовое меню внутри бесконечного цикла while True с ветвлением if-elif-else.

4. Текст программы (Исходный код)

```
staff = [
```

```
    {"фио": "Иванов Иван Иванович", "должность": "Программист", "отдел": "IT",
"зарплата": 120000},
    {"фио": "Петров Петр Петрович", "должность": "Тестировщик", "отдел": "IT",
"зарплата": 80000},
    {"фио": "Сидорова Анна Сергеевна", "должность": "Бухгалтер", "отдел":
"Бухгалтерия", "зарплата": 95000},
    {"фио": "Кузнецов Олег Игоревич", "должность": "Программист", "отдел":
"IT", "зарплата": 150000},
    {"фио": "Смирнова Елена Викторовна", "должность": "HR-менеджер", "отдел":
"Кадры", "зарплата": 70000}
]

def add_worker(spisok): # Добавление нового сотрудника с защитой от ввода букв
    fio = input("Введите ФИО: ")
    pos = input("Введите должность: ")
    dept = input("Введите отдел: ")

    try:
        sal = int(input("Введите зарплату (числом): "))
        new_worker = {"фио": fio, "должность": pos, "отдел": dept, "зарплата":
sal}
        spisok.append(new_worker)
        print("Сотрудник успешно добавлен!")
    except ValueError:
        print("Ошибка: Зарплата должна быть числом! Сотрудник не добавлен.")

def unique_positions(spisok): # Уникальность должностей
    position = set() # Хранение в неупорядоченном виде/ убрать дубликаты
    for i in spisok:
        position.add(i['должность'])
    print("Уникальные должности:", position)

def show_all(spisok): # Показать всех сотрудников
    for worker in spisok:
        print(f"{worker['фио']} | {worker['должность']} | {worker['отдел']} |
{worker['зарплата']} руб.")

def samiy_cool(spisok): # Самый высокооплачиваемый сотрудник
    max_staff = spisok[0]
    for i in spisok:
        if i['зарплата'] > max_staff['зарплата']:
            max_staff = i
    print(f"Самый высокооплачиваемый: {max_staff['фио']}
({max_staff['зарплата']} руб.)")

def sr_zp(spisok): # Рассчёт средней з/п
    total = 0
    for worker in spisok:
        total = total + worker['зарплата']

    a_total = total / len(spisok)
    print(f"Средная зарплата: {a_total}")

def sort_zp(spisok): # Сортировка по зп
    sorted_staff = sorted(spisok, key=lambda x: x['зарплата'])
    for i in sorted_staff:
        print(f"{i['фио']}: {i['зарплата']} руб.")

def poisk_it(spisok): # Поиск сотрудника по отделу и вывод его аамилии
    otd = input("Введите отдел: ")
    naideno = False
    for worker in spisok:
```

```
        if worker['отдел'] == otd:
            print(f"{worker['фио']}       |       {worker['должность']}       |
{worker['отдел']} | {worker['зарплата']} руб.")
            naideno = True
    if naideno == False:
        print('Такого отдела нет или в нём нет сотрудников.')

def filtr_zp(spisok): # Фильтрация по зарплате
    try:
        porog = int(input("Показать сотрудников с зарплатой больше или равной:
"))
        naideno = False
        for worker in spisok:
            if worker['зарплата'] >= porog:
                print(f"{worker['фио']}      |      {worker['должность']}      |
{worker['отдел']} | {worker['зарплата']} руб.")
                naideno = True
        if naideno == False:
            print("Нет сотрудников с такой высокой зарплатой.")
    except ValueError:
        print("Ошибка: Нужно ввести число.")

while True: # Главное меню программы
    print("\n===== МЕНЮ =====")
    print("1. Посчитать среднюю зарплату")
    print("2. Поиск сотрудников по отделу")
    print("3. Добавить нового сотрудника")
    print("4. Все сотрудники")
    print("5. Самый высокооплачиваемый сотрудник")
    print("6. Уникальность должностей")
    print("7. Сортировка по зп")
    print("8. Фильтрация по зарплате")
    print("0. Выход")

    vibor = input("Выбери пункт: ")

    if vibor == "1":
        sr_zp(staff)
    elif vibor == "2":
        poisk_it(staff)
    elif vibor == "3":
        add_worker(staff)
    elif vibor == "4":
        show_all(staff)
    elif vibor == "5":
        samiy_cool(staff)
    elif vibor == "6":
        unique_positions(staff)
    elif vibor == "7":
        sort_zp(staff)
    elif vibor == "8":
        filtr_zp(staff)
    elif vibor == "0":
        print("Выход из программы")
        break
    else:
        print("Неверный пункт меню!")
```

5. Результаты тестирования программу

В соответствии с требованиями раздела 12 методички программа была успешно протестирована на пяти обязательных сценариях:

1) Тест 1: Исходный список staff содержит 5 заполненных словарей. При выборе пункта меню "4" выводятся все 5 сотрудников без ошибок (см. Рисунко 1.1).



Рисунок 1.1 - Обычный набор данных из 5 объектов

2) Тест 2: В исходных данных сотрудники Иванов и Кузнецов имеют одинаковую должность ("Программист"), а сотрудники Иванов, Петров и Кузнецов работают в одном отделе ("IT"). При вызове функции unique_positions должность "Программист" вывелась на экран строго один раз: {'Бухгалтер', 'Программист', 'HR-менеджер', 'Тестировщик'}. Дубликаты корректно удалены (см. Рисунок 1.2).

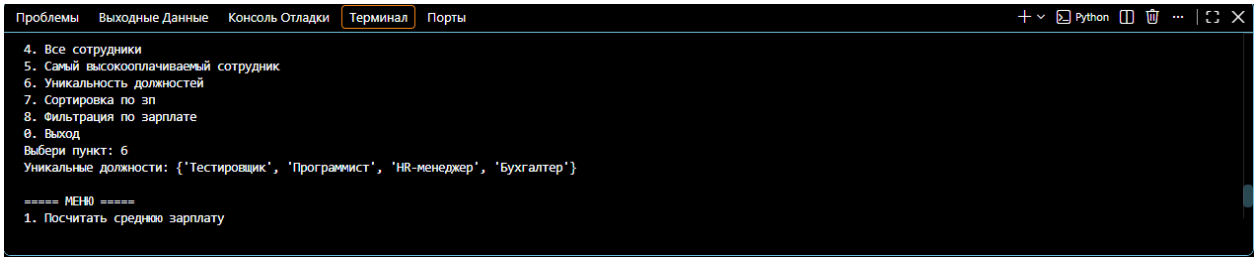


Рисунок 1.2 - Наличие объектов с одинаковыми параметрами

3) Тест 3: При добавлении нового сотрудника (пункт 3) в поле зарплаты намеренно вводились некорректные строковые данные ("сто тысяч"). Блок try-except успешно перехватил ошибку ValueError, вывел предупреждение на экран, и программа продолжила работу в штатном режиме, защитив структуру данных от падения (см. Рисунок 1.3).

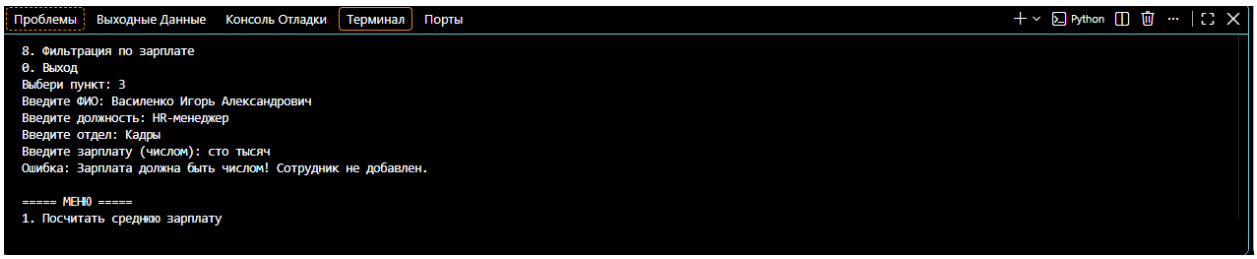


Рисунок 1.3 - Граничные значения и проверка ввода

4) Тест 4: В пункте меню "2" (Поиск по отделу) было введено несуществующее название отдела: "Маркетинг". Программа отработала логику флага и корректно вывела сообщение: Такого отдела нет или в нём нет сотрудников (см. Рисунок 1.4).



Рисунок 1.4 - Поиск при отсутствии подходящих объектов

5) Тест 5: В пункте меню "8" (Фильтрация по зарплате) был введён заведомо завышенный числовой порог: 500000. Условие фильтрации ни разу не выполнилось, программа не упала и выдала текстовое сообщение: Нет сотрудников с такой высокой зарплатой (см. Рисунок 1.5).



Рисунок 1.5 - Фильтрация при отсутствии подходящих объектов

6. Выводы по работе

В ходе выполнения лабораторной работы были приобретены практические навыки работы со сложными структурами данных (списками словарей) в языке Python. Были успешно освоены механизмы циклической обработки коллекций, фильтрации по условиям, поиска экстремумов (максимумов) и агрегации числовых данных (подсчет суммы и среднего значения). На практике закреплены принципы функционального программирования (передача коллекций в функции через именованные параметры) и обработки исключительных ситуаций try-except.